

Machine Learning Basics - Study Notes

Week 1 to Week 4 revision notes | Course: Intro to Data Science (DS101)

1. What is machine learning?

Machine learning (ML) is a way of building programs that learn patterns from data instead of following hand-written rules. Arthur Samuel described it in 1959 as giving computers the ability to learn without being explicitly programmed. A model is the learned function that maps inputs (features) to outputs (labels or predictions).

2. Types of machine learning

- **Supervised learning:** the data has labels. Examples: predicting house prices (regression) and detecting spam (classification).
- **Unsupervised learning:** no labels. The model finds structure, for example k-means clustering of customers into groups.
- **Reinforcement learning:** an agent learns by trial and error using rewards, for example a program learning to play a game.

3. Standard training workflow

Our instructor, Dr. Meera Iyer, insisted on this order for every project:

- Collect and clean the data (remove duplicates, handle missing values).
- Split the data: **70% training, 15% validation, 15% test**. The test set is touched only once, at the very end.
- Choose a model and train it on the training set.
- Tune hyperparameters using the validation set.
- Report the final score on the test set.

4. Overfitting and underfitting

Overfitting means the model memorises the training data and performs badly on new data (low training error, high validation error). **Underfitting** means the model is too simple to capture the pattern (high error on both). Ways to reduce overfitting: get more data, use regularisation (L1 or L2), use dropout in neural networks, use early stopping, or use k-fold cross-validation (we used $k = 5$ in class).

5. Evaluation metrics for classification

Worked example from the lecture: a spam filter was tested on 200 emails. The confusion matrix was:

	Predicted spam	Predicted not spam
Actually spam	TP = 80	FN = 10
Actually not spam	FP = 20	TN = 90

- **Accuracy** = $(TP + TN) / \text{total} = 170 / 200 = \mathbf{0.85}$.

- **Precision** = $TP / (TP + FP) = 80 / 100 = \mathbf{0.80}$. Of the emails flagged as spam, 80% really were spam.
- **Recall** = $TP / (TP + FN) = 80 / 90 = \mathbf{0.889}$. The filter caught 88.9% of the real spam.
- **F1 score** = $2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall}) = \mathbf{0.842}$.

Accuracy alone can mislead when classes are imbalanced. For example, if only 1% of transactions are fraud, a model that always predicts 'not fraud' scores 99% accuracy but has 0% recall.

6. Linear regression and gradient descent

Linear regression predicts a number using $y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + b$. The model is trained by minimising the mean squared error (MSE). **Gradient descent** does this by repeatedly moving the weights a small step against the gradient of the loss. The step size is the **learning rate**. If it is too large the loss bounces around or diverges; if it is too small training is very slow. In our lab we used a learning rate of **0.01** and **500 iterations**, and always scaled the features first (standardisation: subtract the mean, divide by the standard deviation).

7. Class project: house price prediction

- Dataset: **1,200 rows** and **8 features** (area in square feet, bedrooms, bathrooms, age of house, distance to city centre, and others).
- Baseline: predicting the average price gave a test RMSE of about 96,000.
- Linear regression with scaled features reduced test RMSE to about **41,500**.
- A random forest with 200 trees reached about **33,800**, but it took longer to train and was harder to explain.
- Most important feature according to the random forest: **area**, followed by **distance to city centre**.
- Lesson learned: a team lost marks for tuning on the test set. Always tune on the validation set.

8. Quick revision list

- Features are inputs; labels are the correct outputs.
- Never evaluate on the data you trained on.
- High training score with a low validation score means overfitting.
- Use precision when false alarms are costly and recall when misses are costly.
- Scale features before using gradient descent.
- Exam date for DS101: **14 December**. Open-notes, 90 minutes, 4 questions.